



WeCloudData

WEEK 6 · MODEL DEPLOYMENT & SQL

SQL Foundations for Data Science





Where SQL Sits in the ML Pipeline

WHY THIS WEEK

You've built models on data that was already clean, joined, and in one table. In real world, that table doesn't exist yet: it lives across a database, and SQL is how you build it, before it ever reaches a DataFrame.

1

RAW TABLES

Orders, customers, products,
returns: separate, normalized, full of
noise

2

SQL

Filter, aggregate, join, engineer
features: directly at the source

3

TRAINING SET

One clean, feature-rich table: ready
for `pandas.read_sql()`



The Superstore Schema

TODAY'S DATASET

Four tables. Think of orders as your user behavior log, and products as your item catalog: exactly the shape of data you'll join for any recommender, churn, or demand-forecasting model.

orders

OrderID, ProductID, CustomerID, OrderDate, OrderPriority, OrderQuantity, Sales, Discount, ShipMode, Profit, UnitPrice, ShippingCost

behavior log

customers

CustomerID, CustomerName, Province, Region, CustomerSegment

user profile

products

ProductID, ProductName, ProductCategory, ProductSubCategory, ProductContainer, ProductBaseMargin

item catalog

returns

OrderID, Status

label / outcome



WeCloudData

BEFORE THE MODULES

SQL Basics

The core commands and join types you will use all week, in their simplest form.



DDL, DML, and DCL

THREE CATEGORIES OF SQL COMMANDS

DDL

Data Definition Language

Defines and changes table structure

CREATE TABLE

ALTER TABLE

DROP TABLE

TRUNCATE TABLE

DML

Data Manipulation Language

Reads and changes the data itself

SELECT

INSERT

UPDATE

DELETE

DCL

Data Control Language

Controls who can access what

GRANT

REVOKE



SELECT

DML: RETRIEVE DATA

```
SELECT column1, column2, ...  
FROM table_name  
WHERE condition;
```

-- Example

```
SELECT CustomerName, Region  
FROM customers  
WHERE Region = 'West';
```

WHAT IT DOES

Reads rows from a table without changing them.

Choose specific columns by name, or use * for all columns (avoid * on large tables).

WHERE filters which rows come back.

The most common statement you will write all week.



INSERT

DML: ADD A ROW

```
INSERT INTO table_name (col1, col2, ...)
VALUES (val1, val2, ...);
```

-- Example

```
INSERT INTO customers
  (CustomerID, CustomerName, Region)
VALUES
  (10001, 'Aisha Al-Farsi', 'Central');
```

WHAT IT DOES

Adds a new row to a table.

The column list is optional, but it keeps the statement readable and safe.

Values must match the column order and data types.

One INSERT can add a single row, or many rows at once.



UPDATE

DML: CHANGE EXISTING ROWS

```
UPDATE table_name  
SET column1 = value1, column2 = value2  
WHERE condition;
```

-- Example

```
UPDATE orders  
SET OrderPriority = 'Critical'  
WHERE Sales > 5000;
```

WHAT IT DOES

Changes values in rows that already exist.

WHERE decides which rows are touched. Leaving it off updates every row.

Always test the WHERE clause with a SELECT first.

Common in cleanup work: standardizing values, fixing typos, backfilling defaults.



DELETE

DML: REMOVE ROWS

```
DELETE FROM table_name  
WHERE condition;  
  
-- Example  
DELETE FROM orders  
WHERE OrderDate < '2009-01-01';
```

WHAT IT DOES

Removes whole rows from a table.

WHERE decides which rows are removed. Leaving it off deletes every row.

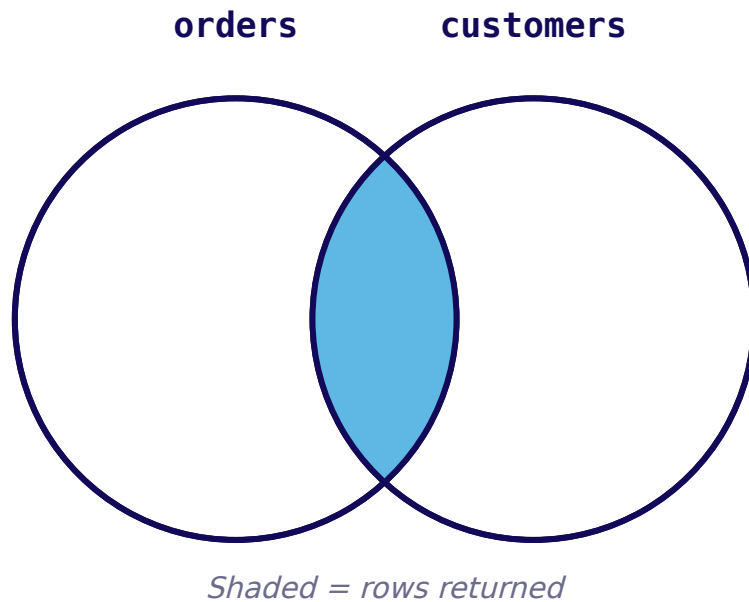
Deleted rows are gone. There is no undo without a backup.

In practice: archive or soft-delete with a status flag before a hard DELETE.



INNER JOIN

1 OF 4 · MATCHING ROWS ONLY



Returns only the rows where the join key matches in both tables. An order with no matching customer is dropped from the result.

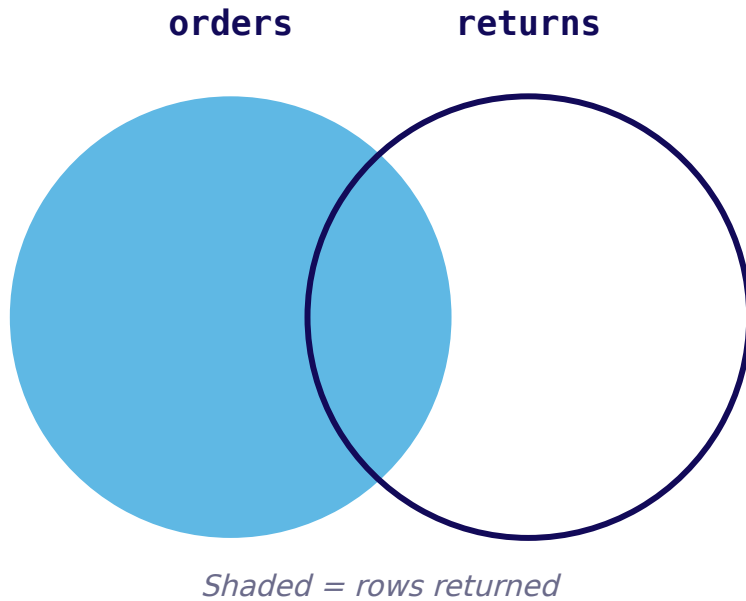
```
SELECT o.OrderID, c.CustomerName  
FROM orders o  
INNER JOIN customers c  
    ON o.CustomerID = c.CustomerID;
```

Use INNER JOIN when a missing match means the row is not usable for the task at hand.



LEFT JOIN

2 OF 4 · EVERY ROW ON THE LEFT



Keeps every row from the left table. Where there is no match on the right, those columns come back as NULL.

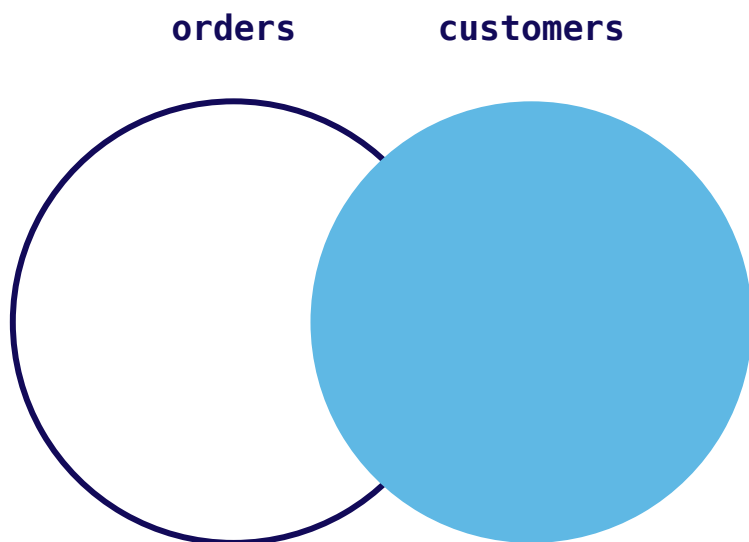
```
SELECT o.OrderID, r.Status  
FROM orders o  
LEFT JOIN returns r  
  ON o.OrderID = r.OrderID;
```

A NULL Status here means the order was never returned. That NULL is a feature, not a gap.



RIGHT JOIN

3 OF 4 · EVERY ROW ON THE RIGHT



Shaded = rows returned

The mirror of LEFT JOIN. Keeps every row from the right table, and fills in NULL where the left table has no match.

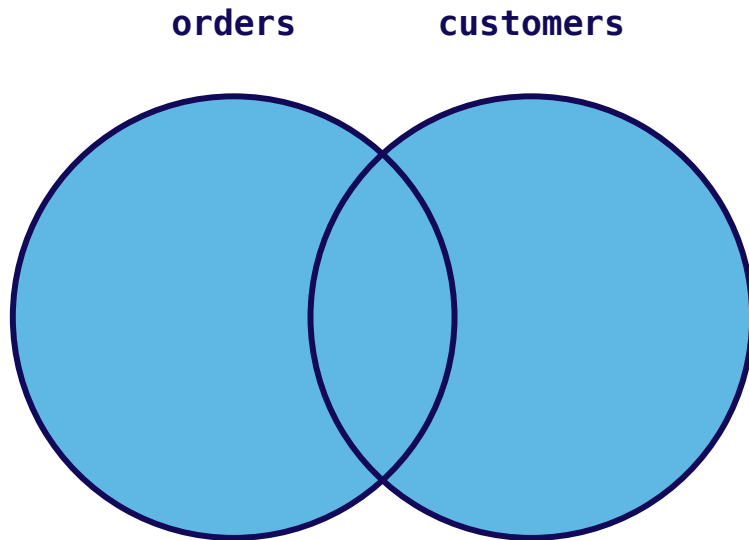
```
SELECT o.OrderID, c.CustomerName  
FROM orders o  
RIGHT JOIN customers c  
  ON o.CustomerID = c.CustomerID;
```

Rarely needed in practice: most RIGHT JOINs can be rewritten as a LEFT JOIN by swapping the table order.



FULL OUTER JOIN

4 OF 4 · EVERY ROW FROM BOTH SIDES



Shaded = rows returned

Keeps every row from both tables. Where a row has no match on the other side, those columns come back as NULL.

```
SELECT o.OrderID, c.CustomerName  
FROM orders o  
FULL OUTER JOIN customers c  
  ON o.CustomerID = c.CustomerID;
```

MySQL has no native FULL OUTER JOIN. It is usually built from a LEFT JOIN combined with a RIGHT JOIN using UNION.



WeCloudData

MODULE 1

Basic Querying & Filtering

SELECT, WHERE, ORDER BY, LIMIT, DISTINCT: the first pass of cleaning a dataset happens before Pandas even sees it.



WHERE, ORDER BY, LIMIT

CLAUSES YOU WILL USE IN THIS MODULE

WHERE

Filters which rows come back

Conditions: =, !=, >, <, >=, <=,
BETWEEN, IN, LIKE, IS NULL

```
WHERE Sales > 1000
```

```
WHERE Region IN  
( 'West', 'East' )
```

```
WHERE ShipMode LIKE 'Air%'
```

ORDER BY

Sorts the result set

ASC (default) or DESC, on one or more
columns

```
ORDER BY Sales DESC
```

```
ORDER BY Region, Sales DESC
```

LIMIT

**Caps how many rows come
back**

Pair it with ORDER BY, or the top rows
are arbitrary

```
LIMIT 10
```

```
ORDER BY Sales DESC
```

```
LIMIT 5
```



MODULE 1

Basic Querying & Filtering

```
SELECT column1, column2
FROM orders
WHERE OrderPriority = 'Critical'
      AND Sales BETWEEN 100 AND 5000
      AND ShipMode <> 'Delivery Truck'
      AND ProductID IN (657768, 293693)
ORDER BY OrderDate DESC
LIMIT 100;
```

```
SELECT DISTINCT CustomerSegment
FROM customers;
```

THE DATA SCIENCE TOUCH

Filter noise before it costs you compute

WHERE with BETWEEN / IN / LIKE removes invalid rows and outliers before a single byte hits memory

LIMIT is your fastest sanity check on a new table: never SELECT * on a full production table

DISTINCT catches duplicate log entries that would silently inflate frequency features

IS NULL / IS NOT NULL is your first missing-data audit: before you write pandas.dropna()



MODULE 1

Basic Querying & Filtering

Pull high-value, recent Critical-priority orders: a candidate slice for a rush-delivery risk model.

```
SELECT OrderID, CustomerID, OrderDate, Sales, OrderPriority
FROM orders
WHERE OrderPriority = 'Critical'
      AND Sales > 1000
      AND OrderDate >= '2011-01-01'
ORDER BY Sales DESC
LIMIT 5;
```

RESULT (excerpt)

OrderID	CustomerID	OrderDate	Sales	OrderPriority
3841	60800210	2011-11-02	8399.98	Critical
6852	51402540	2011-06-14	6720.10	Critical
2214	40732966	2011-03-22	5410.55	Critical



MODULE 2

Aggregations

GROUP BY and HAVING turn thousands of transaction rows into the customer- and product-level features a model actually trains on.



GROUP BY, HAVING

CLAUSES YOU WILL USE IN THIS MODULE

GROUP BY

Collapses rows into one row per group

Group by one or more columns, then aggregate each group with COUNT, SUM, AVG, MIN, or MAX

```
SELECT CustomerID, COUNT(*)  
FROM orders  
GROUP BY CustomerID
```

HAVING

Filters groups after they are aggregated

WHERE filters rows before grouping. HAVING filters groups, and can reference an aggregate like COUNT(*) or SUM(Sales)

```
GROUP BY CustomerID  
HAVING COUNT(*) >= 3
```



MODULE 2

Aggregations

```
SELECT CustomerID,  
       COUNT(OrderID) AS order_count,  
       SUM(Sales) AS total_spend,  
       AVG(Sales) AS avg_order_value,  
       MAX(OrderDate) AS last_order_date  
FROM orders  
GROUP BY CustomerID  
HAVING COUNT(OrderID) >= 3  
ORDER BY total_spend DESC;
```

THE DATA SCIENCE TOUCH

This is feature engineering, in SQL

One row per customer, one column per feature: the exact shape a model's feature table needs

COUNT / SUM / AVG per group = frequency, monetary, and average-basket features, computed at the source

HAVING filters on the aggregate itself: e.g. drop customers with too few orders to trust a trend from

COALESCE(SUM(Discount), 0) turns a NULL aggregate into a real, model-safe zero



MODULE 2

Aggregations

Aggregate to the product level: a feature table ready to feed a pricing or margin model.

```
SELECT ProductID,  
       COUNT(*)      AS times_ordered,  
       SUM(Profit)    AS total_profit,  
       ROUND(AVG(Discount), 2) AS avg_discount  
FROM orders  
GROUP BY ProductID  
HAVING SUM(Profit) < 0  
ORDER BY total_profit ASC;
```

RESULT (excerpt)

ProductID	times_ordered	total_profit	avg_discount
657768	42	-318.40	0.18
293693	27	-201.75	0.22
481920	19	-95.10	0.15



MODULE 3

Joins & Unions

The training set doesn't live in one table. Joins are how a behavior log, a catalog, and a customer profile become a single modeling dataset.



MODULE 3

Joins & Unions

```
SELECT o.OrderID, o.Sales, c.CustomerSegment,  
       p.ProductCategory, r.Status  
FROM orders o  
INNER JOIN customers c  
    ON o.CustomerID = c.CustomerID  
LEFT JOIN products p  
    ON o.ProductID = p.ProductID  
LEFT JOIN returns r  
    ON o.OrderID = r.OrderID;  
  
-- r.Status IS NULL => 'never returned'
```

THE DATA SCIENCE TOUCH

A LEFT JOIN's NULL is a feature, not an error

INNER JOIN keeps only rows that match both sides: use it when a missing link means the row isn't usable

LEFT JOIN keeps every row on the left even with no match: the NULL often is the label itself

UNION ALL stacks similarly-shaped queries into one training set without deduping; UNION removes duplicates when that's wanted

An unintended fan-out silently duplicates rows and corrupts every aggregate downstream



MODULE 3

Joins & Unions

One row per order, enriched with customer segment, product category, and return outcome.

```
SELECT o.OrderID, o.OrderDate, o.Sales, o.Discount,  
       c.CustomerSegment, p.ProductCategory,  
       CASE WHEN r.Status IS NULL THEN 0 ELSE 1 END AS was_returned  
FROM orders o  
JOIN customers c ON o.CustomerID = c.CustomerID  
JOIN products p ON o.ProductID = p.ProductID  
LEFT JOIN returns r ON o.OrderID = r.OrderID;
```

RESULT (excerpt)

OrderID	Sales	CustomerSegment	ProductCategory	was_returned
8710	151.35	Corporate	Office Supplies	0
5251	3821.04	Corporate	Technology	1
3841	8399.98	Home Office	Furniture	0



Why This Matters for Your Models

3+

tables typically joined to build one real-world training set

70%

of a data scientist's time is data prep: most of it belongs in SQL, not Pandas

0

rows you should filter, aggregate, or join twice: do it once, at the source